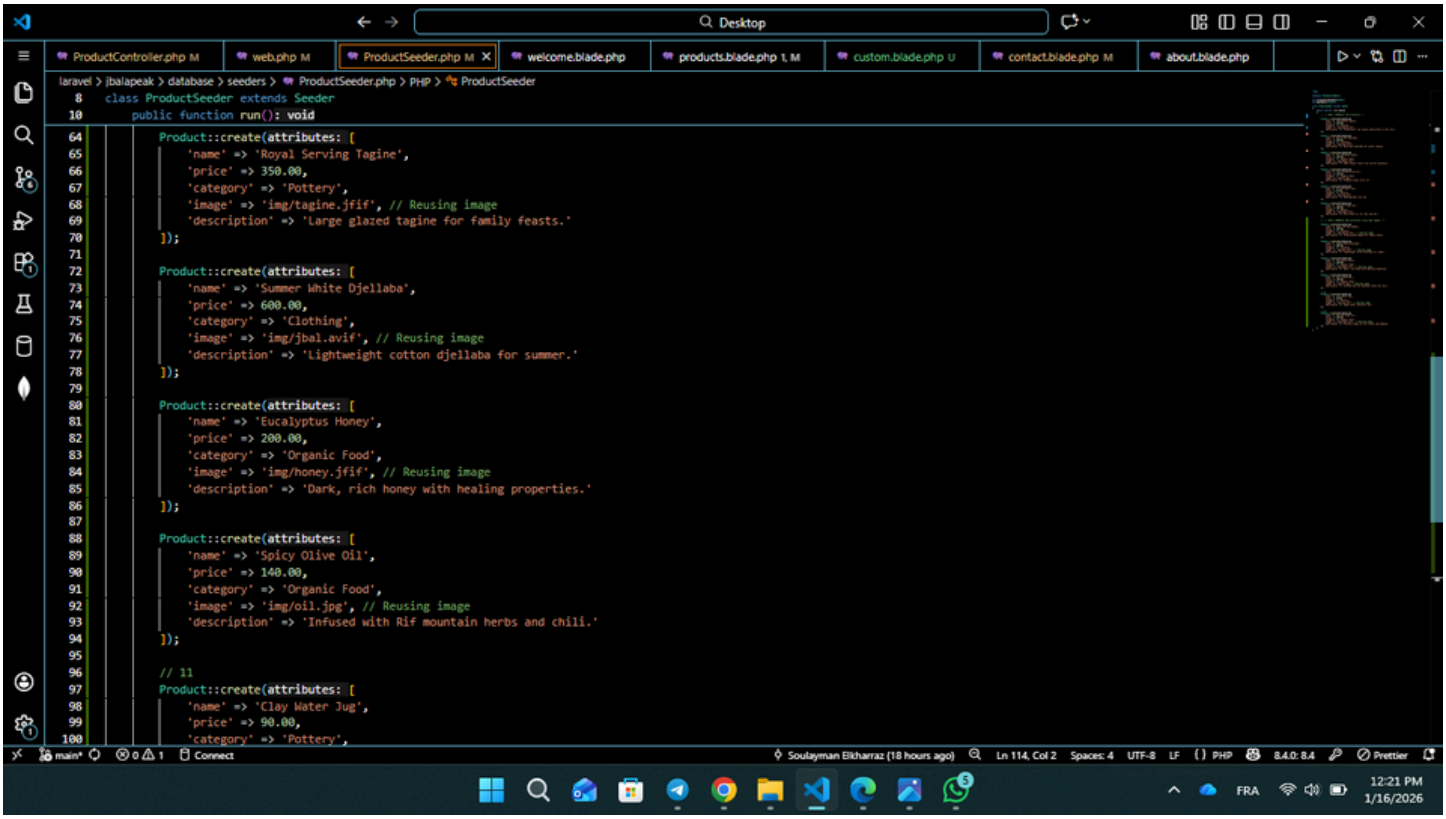
RAPPORT : Mise en place de la Pagination (Atelier 8)

1. Objectif de l'Atelier

L'objectif est d'optimiser l'affichage des produits en intégrant un système de **pagination**. Au lieu de charger tous les produits sur une seule page, nous les divisons en plusieurs pages (6 produits par page) pour améliorer la performance et l'expérience utilisateur .

2. Préparation des Données (Seeding)

Action : Nous avons modifié le ProductSeeder pour insérer **12 produits** dans la base de données, afin d'avoir suffisamment de données pour générer au moins 2 pages.



3. Modification du Contrôleur (Logique)

Action : Dans ProductController, nous avons remplacé la méthode all() par la méthode paginate(6). Cela permet à Laravel de gérer automatiquement le calcul des pages et les requêtes SQL LIMIT/OFFSET.

- **Code :** $products = Product::paginate(6);

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Product;

3 references | 0 implementations
class ProductController extends Controller
{
1 reference | 0 overrides
public function index(): View
{

    $products = Product::paginate(perPage: 6);
return view(view: 'products', data: ['products' => $products]);
}

    1 reference | 0 overrides
    public function filter($category): View
    {
        $products = Product::where(column: 'category', operator: $category)->paginate(perPage: 6);

        return view(view: 'products', data: [
            'products' => $products,
            'currentCategory' => $category
        ]);
    }
}
```

4. Personnalisation de la Pagination (Vue)

Action : Pour respecter la charte graphique "Jbala Peak" (Vert et Or), nous avons créé une vue de pagination personnalisée au lieu d'utiliser le style par défaut.

- **Fichier créé :** resources/views/vendor/pagination/custom.blade.php

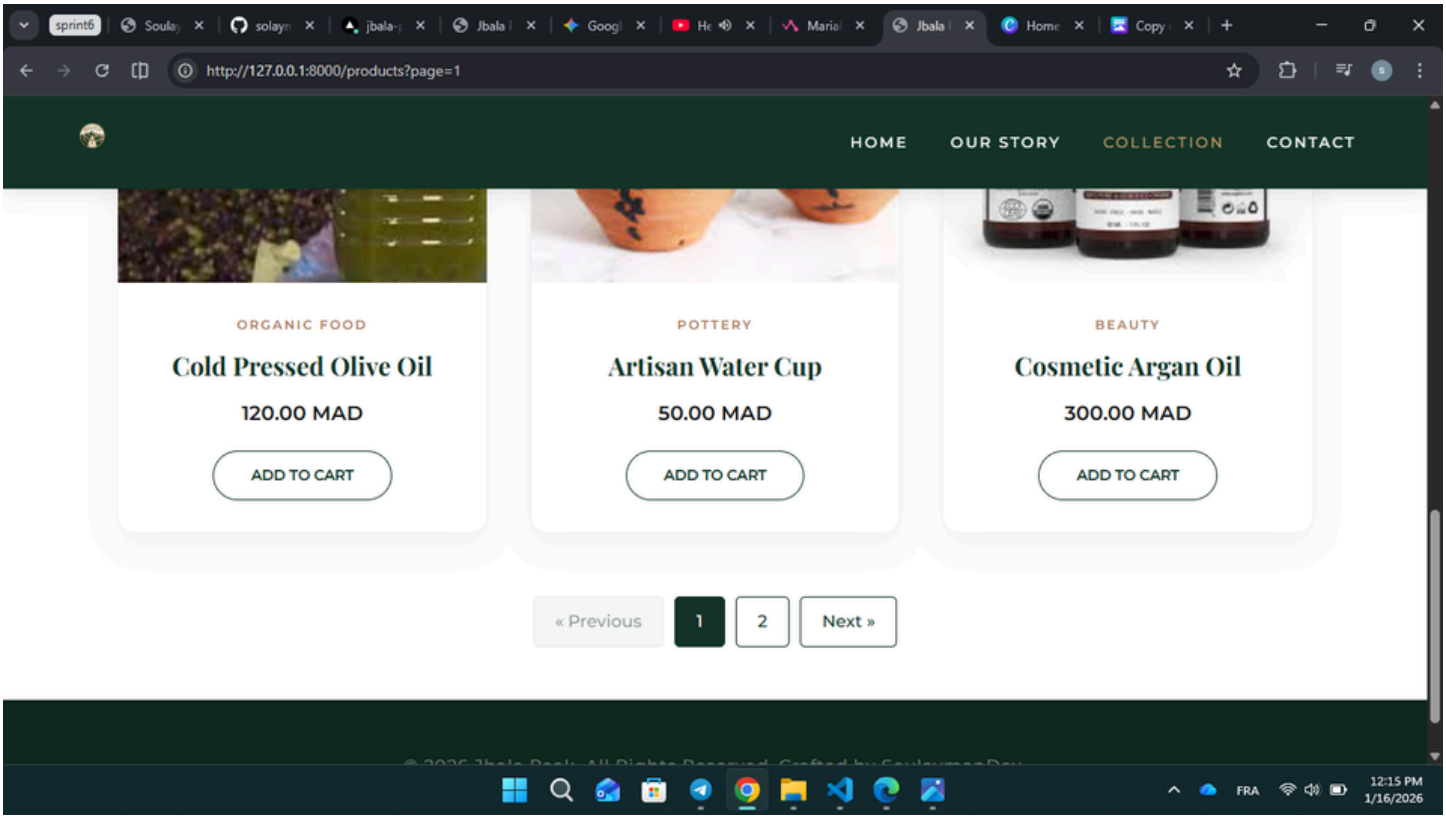
```php
laravel > jbalapeak > resources > views > vendor > pagination > custom.blade.php > ...
@if ($paginator->hasPages())
    <ul class="pagination">
        {{-- Previous Page Link --}}
        @if ($paginator->onFirstPage())
            <li class="page-item disabled">
                <span class="page-link">&laquo; Previous</span>
            </li>
        @else
            <li class="page-item">
                <a class="page-link" href="{{ $paginator->previousPageUrl() }}" rel="prev">&laquo; Previous</a>
            </li>
        @endif

        {{-- Pagination Elements --}}
        @foreach ($elements as $element)
            {{-- "Three Dots" Separator --}}
            @if (is_string(value: $element))
                <li class="page-item disabled"><span class="page-link">{{ $element }}</span></li>
            @endif

            {{-- Array Of Links --}}
            @if (is_array(value: $element))
                @foreach ($element as $page => $url)
                    @if ($page == $paginator->currentPage())
                        <li class="page-item active"><span class="page-link">{{ $page }}</span></li>
                    @else
                        <li class="page-item"><a class="page-link" href="{{ $url }}">{{ $page }}</a></li>
                    @endif
                @endforeach
            @endif
        @endforeach

        {{-- Next Page Link --}}
        @if ($paginator->hasMorePages())
            <li class="page-item">
                <a class="page-link" href="{{ $paginator->nextPageUrl() }}" rel="next">Next &raquo;</a>
            </li>
        @else
            <li class="page-item disabled">
```

5. Intégration dans la Vue Principale

Action : Dans products.blade.php, nous avons appelé la méthode links() en spécifiant notre vue personnalisée.

- **Code :** {{ $products->links('vendor.pagination.custom') }}

```
    <div style="margin-top: 60px; display: flex; justify-content: center;">
        {{ $products->links('vendor.pagination.custom') }}
    </div>
</section>
```

6. Résultat Final

Le site affiche désormais 6 produits par page avec des boutons de navigation fonctionnels et stylisés.